

7/6/21

Page No. _____

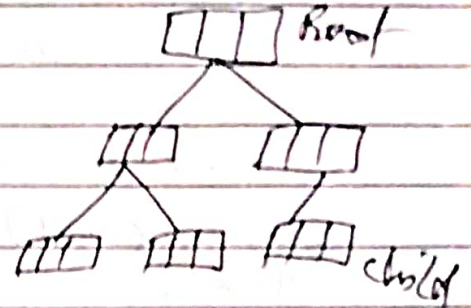
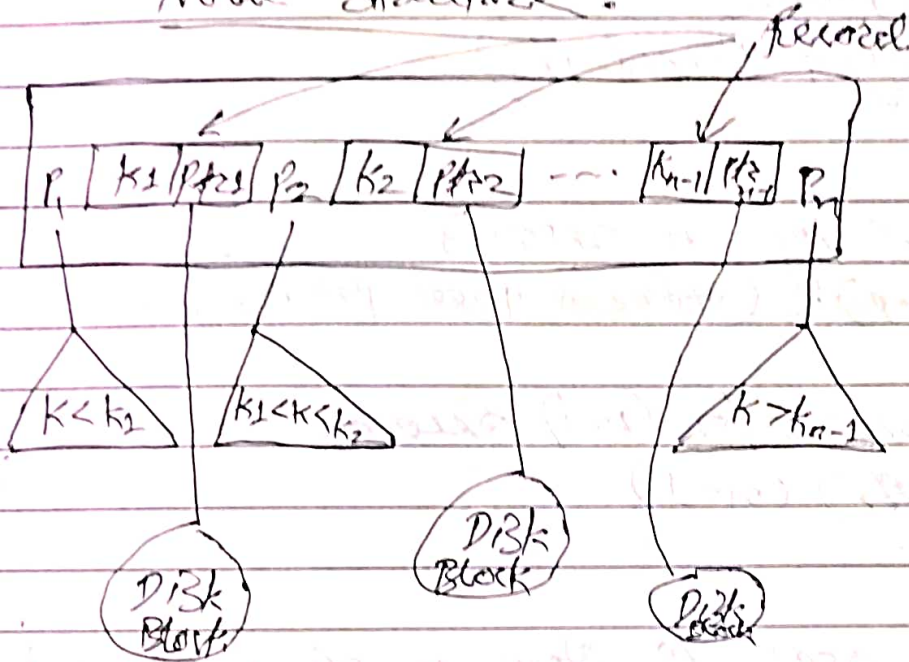
Date: _____

"Implementation of multilevel Index (MI) using B Tree"

1 Node = 1 Block

→ Each block in MI, treated as one node in B Tree.

Node structure:



→ Search start from root

→ $\begin{matrix} \swarrow & \searrow \\ < & > \end{matrix}$
if key value < than
Block move
towards left
B Reverse

$P_1, P_2, \dots, P_n \rightarrow$ Block / Tree / Index pointer

B Tree
→ Provide direct

$k_1 < k_2 \dots k_{n-1} \rightarrow$ keys

$P_{12}, P_{22}, P_{32}, \dots, P_{n-1,2} \rightarrow$ Record / data pointer

Access record
from disk if
found in Block

Block $\approx$ Tree $\approx$ Index
pointer $\approx$ pointer $\approx$ pointer

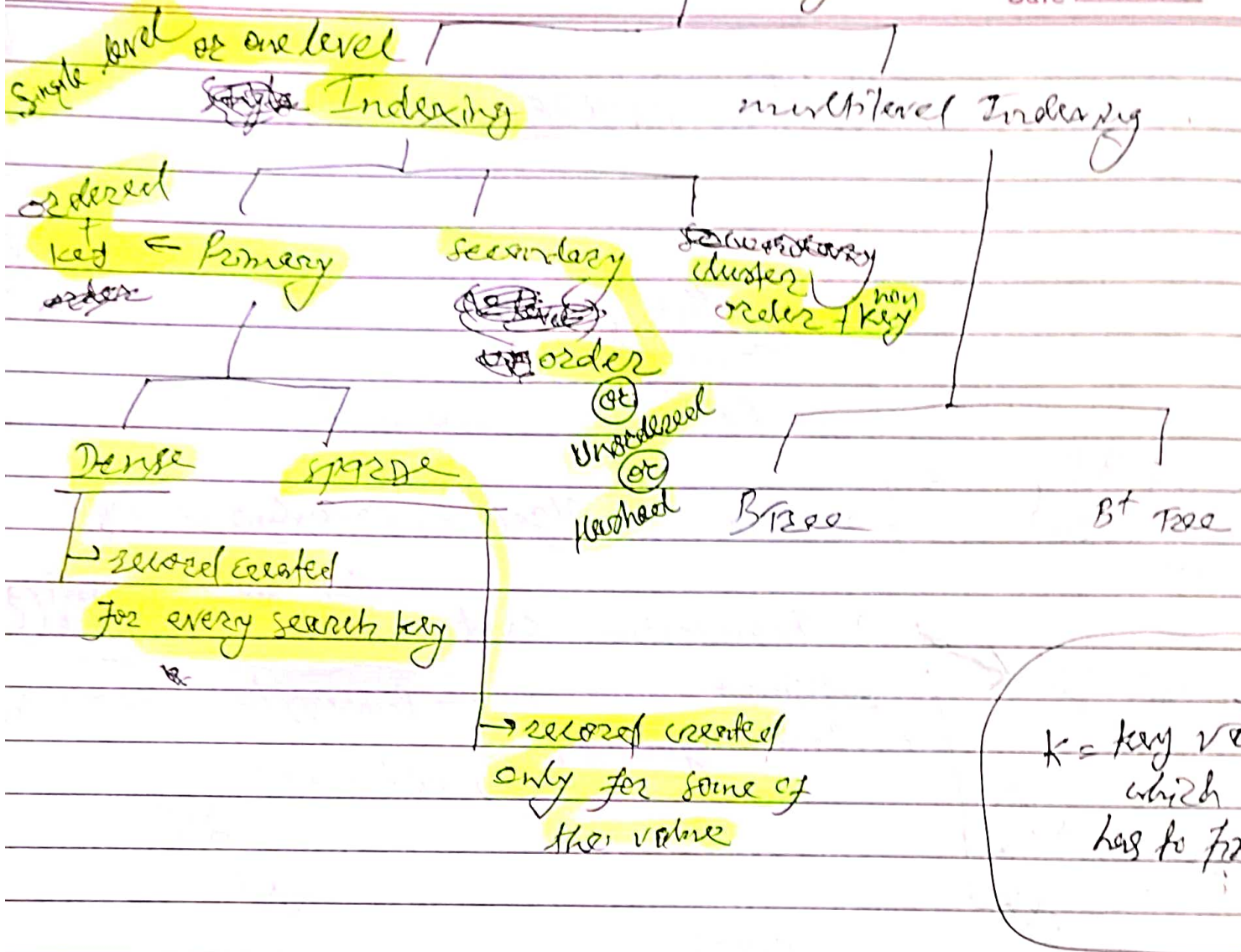
→ B Tree is Balanced
search tree (height)

~~Block~~

Record
pointer $\approx$ Data
pointer

It minimize num(disk)
Access

It is used to locate & Access data quickly
Indexing = A type of data structure
Page No. _____
Date: ____/____/____



Secondary Index (ordered)

2 Variations $\rightarrow$ Candidate key + unique value

$\rightarrow$ Non key with duplicate value

$\rightarrow$ Here data file could be ordered, unordered & may be hashed.

$\rightarrow$ search key is used to loopup records in file.

N records = order of B Tree

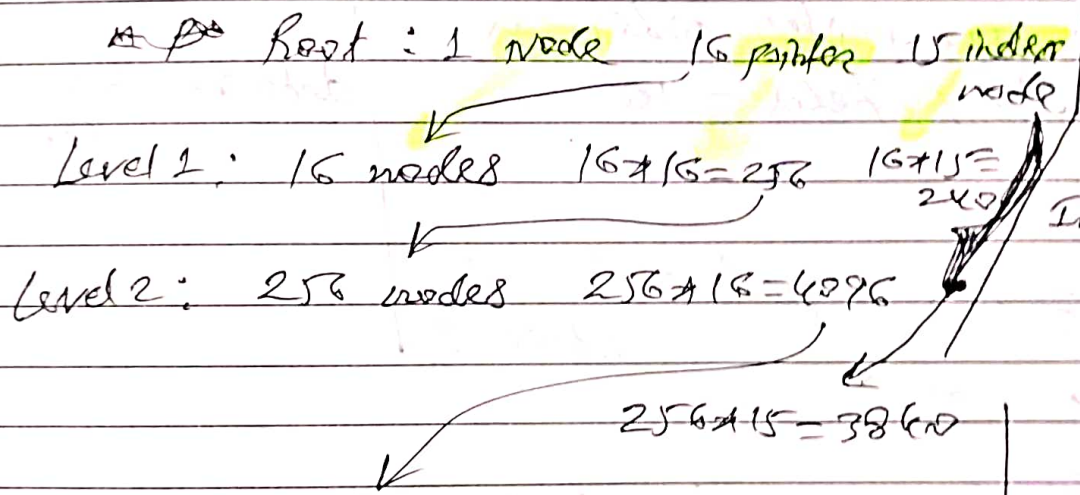
Root = level 0
By default

Ex → calculate the approximate num of Index records that can be stored in B tree of 3 levels, for the above Example.
Assume that each node is 60% full.

order * node capacity

$$\text{num pointers} = \lfloor 24 \times 0.6 \rfloor = \lfloor 16.56 \rfloor = 16 \text{ pointers}$$

num pointers from
 order = order + % full



num pointers then
 Records = num
 Index = (n-1) record

Level 3: 4096 nodes, 4096*16 = 65536
 4096*15 = 61440

num (Index)
 records = $P^{l+1} - 1$

num (Pointer) = P^{l+1}

∴ Total num of Index Records = 15 + 240 + 3840 + 61440
 (Add All Index records of all level)
 = 65535

l = level
~~order~~

P = pointer
 computed
 & order
 multiply with

only valid for % of full node given

Order of B Tree (let 'n')

$\rightarrow$ It is the max num of tree pointers in node.

$\rightarrow n = \text{Tree pointer}$
 $(n-1) = (\text{key} + \text{Record ptr})$

Space req^r for n records
 $(n+1) \times n \quad (\text{order} + \text{Tree pointer})$

Space req^r for (n-1) records
 $(k + \text{ptr}) (n-1)$

order $\therefore$ Total space req^r to store n & (n-1) records

$n+1 + (k + \text{ptr}) (n-1) \leq \text{Block size}$

Ex

Calculate the order of B tree if

$k = \text{key value} = 9B$, $\text{Record pointer} = 7B$,
 $\text{Block pointer} = 6B$, $\text{Block} = 512B$
 (P) (B) size

Solⁿ $\Rightarrow n+6 + (9+7)(n-1) \leq 512$

Lower Bound
 $as \leq B (\text{Block})$

$6n + 16n - 16 \leq 512$
 $+16$

$n \leq \left\lceil \frac{528}{22} \right\rceil = \frac{264}{11} \leq 24 \text{ Byte}$

when % full not given

$$\min(\text{Index}) \text{ records} = 0^L * (0-1)$$

Page No. _____
Date: ____/____/____

Q.10

Soln → Block = 512
size

used Example (1)

$$\text{key} = 9$$

$$n = 24$$

$$P = 6$$

$$ptr = 7$$

node

pointer

Index node

level 0 = Root · 1 node 24 pointer 23 records

level 1: 24 node $2n * 2n = 576$ $24 * 23 = 552$

level 2: 576 node $576 * 24 = 13824$ $576 * 23 = 13248$

min Index

Ans → Records of level 2 = 13248

$$-11- = 0^L * (0-1) + 24^2 * (24-1) = 13248$$

Ex (10)

$$\min(\text{Index}) \text{ of level} = [0^{L+1} - 1] \rightarrow \text{valid only when}$$

$$0 + 6 + (0-1)(9+7) \leq 512$$

$$\therefore \approx 24$$

$$60 + 160 - 16 \leq 512$$

$$= 13823$$

$$220 = 528$$

$$0 = \frac{528}{24} = 22$$

$$0^L * (n-1)$$

Put p where % full given order
otherwise compute with $\rightarrow$ value

Page No.

Date:

$\rightarrow$ The min (index) records in B tree of order 'O' of 'L' levels

$$P = \frac{P^{L+1}}{O - 1}$$

$P = \text{pointer} = O \times \% \text{ full nodes}$

Q.9

Soln $\Rightarrow$ tree pointer order = n
 $k = \text{key field} = 10$

data ptr = record
ptr

$$\text{Disk block} = B = 512$$

ptr = Bptr = 8 $\rightarrow$ final order of B tree

$$p = Bptr = 5$$

$$n \times 5 + (n-1)(10+8) \leq 512$$

$$5n + 18n - 18 \leq 512$$

$$13n \leq 530$$

$$n = \left\lfloor \frac{530}{13} \right\rfloor = 40$$

No direct Access to Disk Block $\therefore$ go down towards leaf

Page No. _____

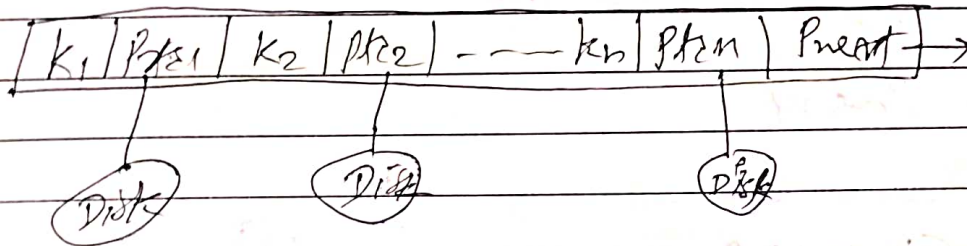
Date: ____/____/____

"Inert" of MI using

B+ Tree (Ordered Access = Search level by level)

$\hookrightarrow$ more efficient than B tree

* Structure of leaf node

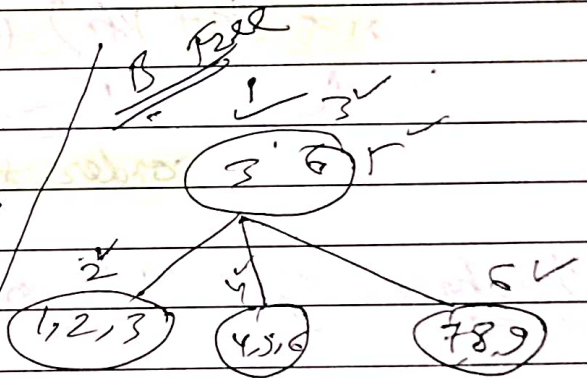


pnext = point to sibling in leaf level

$k_1 < k_2 < \dots < k_n \rightarrow$ keys

$pte_1, pte_2, \dots, pten \rightarrow$ record/data pointer

pnext $\rightarrow$ Block / Tree / Index pointer

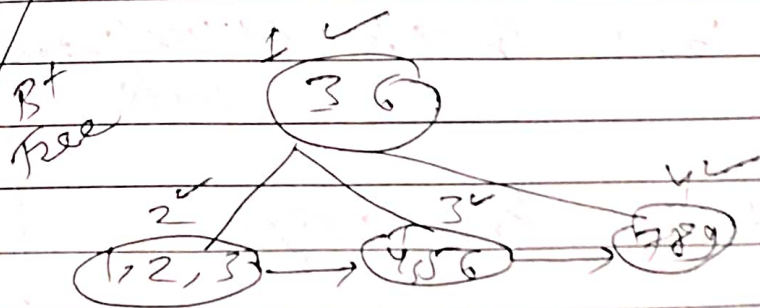


find = 2,

find = 4, 5

find = 7, 8

} To Access All we need 6 Access



find 2, 4-5, 7, 8

To Access using pnext need = 4 Access

All leaf pointer stored or pointer to null

Page No. _____

Date: ____/____/____

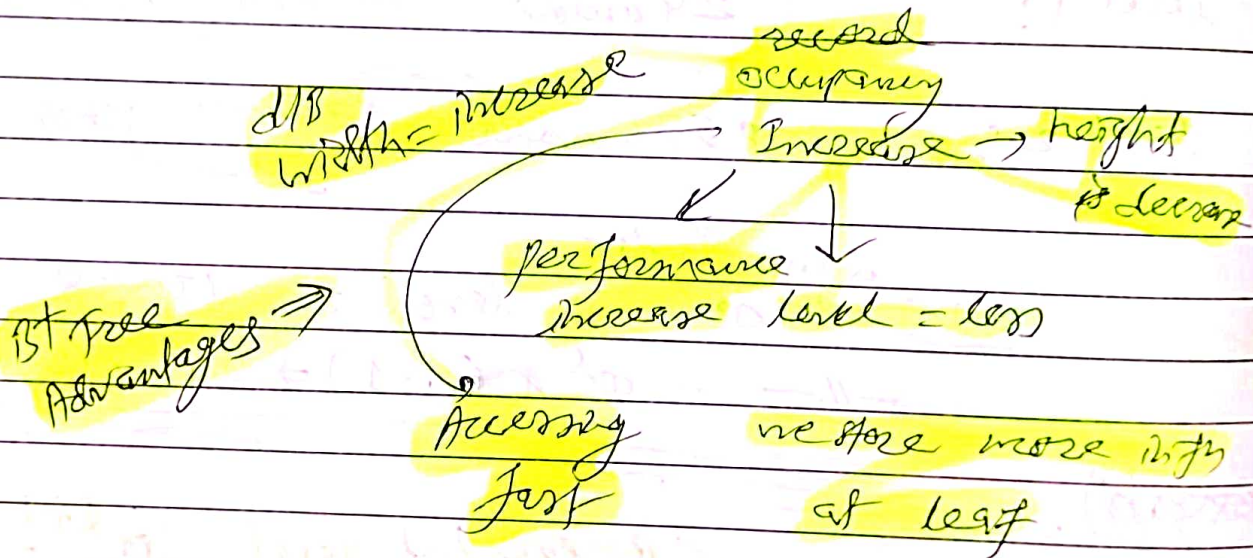
Note

There are 2 modifications ^{suggested} in B tree to make B+ tree,

(1) Internal Node $\Rightarrow$ Remove record pointer
It contains (tree pointer + keys)

(2) leaf node $\Rightarrow$ Remove tree pointer,
It contains (key + record pointer)
Records

$\Rightarrow$ Internal & leaf node have different structure of B+ tree after modification.



order of internal node (let 'n')

It is max num of free pts in a node.

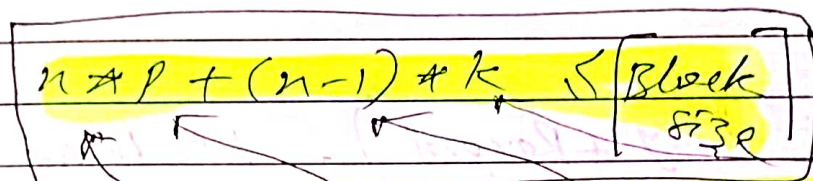
$n \leftarrow$ Free pointer

$(n-1) \leftarrow$ keys

$p \leftarrow$ free/Block/End ptr

$k \leftarrow$ keys

$n \leftarrow$ order



$$\text{order} \times \text{Free pointer} + (\text{order} - 1) \times \text{key} \leq \text{Block size}$$

Ex →

Find the order of B+ tree

if $k=9$, $p=7$

$p_{\text{next}}=6$, $B=512$

Soln → order of Internal node

$$n \times 6 + (n-1) \times 9 \leq 512$$

$$6n + 9n - 9 \leq 512$$

$$15n \leq 521$$

$$n \leq \left\lfloor \frac{521}{15} \right\rfloor = \underline{\underline{34}}$$

order of leaf node

$$n \times (9+7) + 6 \leq 512$$

$$16n \leq 512 - 6$$

$$n = \frac{506}{16} = \underline{\underline{31}}$$

max (num)
of Index records

Page No. _____
Date: ____/____/____

Order of leaf node (let 'n')

→ It is the max num of Index records
in a node.

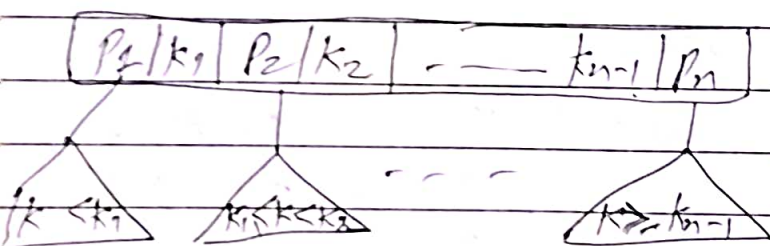
$$n = \text{key} + \text{Record ptr}$$

order $\frac{1}{n} = P_{next}$

$$n * (k + p_{tr}) + P_{next} \leq \left[\text{Block size} \right]$$

order * Record + next ≤ Block size

Structure of Internal Node



$P_1, P_2, \dots, P_n \rightarrow$ Block Tree/Index pointer

$k_1 < k_2 < \dots < k_{n-1} \rightarrow$ keys

if key found
then also
go to down
towards leaf

to access
Block from child

in case of
no direct Access

Ex $\Rightarrow$ Calculate the num of index records that can be stored in B+ Tree of 3 levels for the above questions.
Assume that each node is 50% full.

Solⁿ $\Rightarrow$ order of leaf node = $34069 = 21$

" Internal node = $34 + 0.69 = 23$

Root: 1 node 23 pointers

level 1: 23 $\swarrow$ $23 \times 23 = 529$

level 2: 529 $\swarrow$ $529 \times 23 = 12167$

level 3: 12167 $\swarrow$ $12167 \times 21 = 255507$

leaf of
leaf
:

Direct formula to compute

Total Num (Index) Records = (Internal node order)^{level} * leaf node order

B Tree

$$n \text{ pointers}, (n-1) \text{ Index} + \text{Record} \quad \text{+ (key + Record index)}$$
B+ Tree

$$n + \text{ (key + Record) ptr}, 1 \rightarrow \text{Point (Block)}$$
B+ Tree

$$\text{Internal Node} \quad n \leftarrow \text{Tree pointer}, (n-1) \text{ keys}$$

Note $\Rightarrow$ Record pointer / data pointer is eliminated in Internal nodes of B+ Tree

Q. 16]

$$8017 \Rightarrow k = 12$$

$$B = 1024$$

$$P_1 = 10 \quad k = 3$$

$$P = 8$$

$$n \times 8 + (n-1) \times 12 \leq 1024$$

$$8n + 12n - 12 \leq 1024$$

$$20n \leq 1036$$

$$\text{Order} = n = \left\lfloor \frac{1036}{20} \right\rfloor = 51$$

$$\text{max (search keys)} = \text{order} - 1 = 51 - 1 = 50$$

$$(n-1)$$

(Pkt)
If Record pointer
is not given

Page No. _____

Date: / /

then calculate

Internal order
of B+ tree

Q.14

Soln $\Rightarrow$ $k = 8$ $n = ?$

$$B = 512$$

$$P_{\text{ptr}} = 2$$

$$n \times P + (n-1)k \leq B$$

Internal
order
question

$$2n + (n-1)8 \leq 512$$

$$2n + 8n - 8 \leq 512$$

$$10n \leq 520$$

$$n = 52$$

Q.15

Soln $\Rightarrow$ $B = 1024$ $k = 1024$ B

$$P_{\text{ptr}} = 7$$

$$P = 6$$

$$k = 9$$

next

$$n \times (k + P_{\text{ptr}}) + P_{\text{next}} \leq B$$

$$n(7+9) + 6 \leq 1024$$

$$\Rightarrow 16n \leq 1024 - 6 \Rightarrow n = \frac{1018}{16} = 63$$

"B tree Insertion"

order of B-tree = n

max $\rightarrow n$ pointers,
($n-1$) keys

min $\rightarrow \left\lceil \frac{n}{2} \right\rceil$ pointers
 $\left\lceil \frac{n}{2} \right\rceil - 1$ keys

Excerpt for Root

Ex $\rightarrow$ B-tree
order = 5

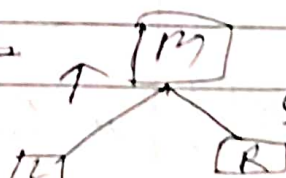
max $\rightarrow 5$ pointer
($5-1$) = 4 keys

min $\rightarrow \left\lceil \frac{5}{2} \right\rceil$ pointer = 3 pointers

$\left\lceil \frac{5}{2} \right\rceil - 1 = 3 - 1 = 2$ keys

Note

* Inserting keys is always into leaf node
& if whenever a node is full while inserting of B-tree
then we split the node into 2 nodes
moving upward directly the middle element.
& all the key less than middle
less than middle



50%

m goes to upper side
(higher level)

100%
[L | m | R]

B-Tree

① Both leaf & internal node store keys & data.

② Data Access from Both leaf & internal node.

③ Less efficient for Range queries

④ More complex for insert & delete

⑤ All nodes are stored
∴ less efficient for space/memory

Page No. _____
Date: ____/____/____

BT Tree

① only leaf Node store data, and internal nodes only stores key

② Data can only be accessed by traversing to leaf nodes

③ more efficient for Range queries

④ More efficient ^{for insert/delete} as data resides only in leaf,
∴ reduce need of re-organizing.

⑤ More space-efficient, as data is stored only at leaf node

go to the left subtree if greater than the middle, move to the right subtree

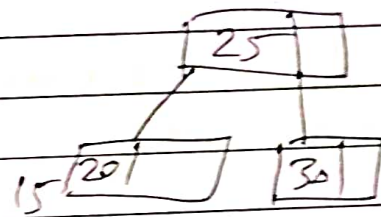
100%
 mem (nodes ~~not~~ full) = 0

Root node = 25

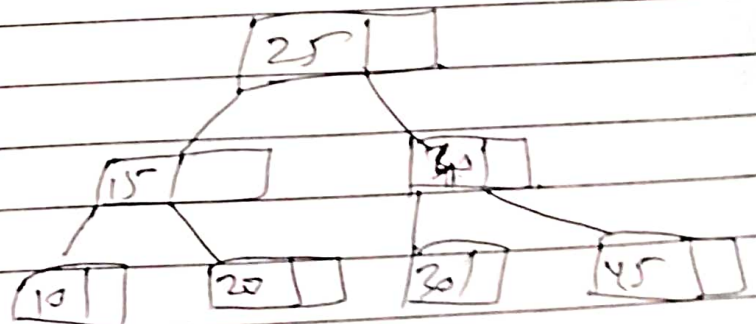
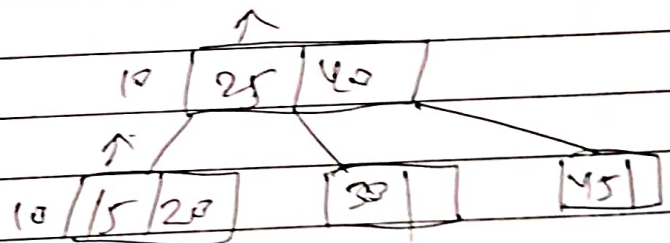
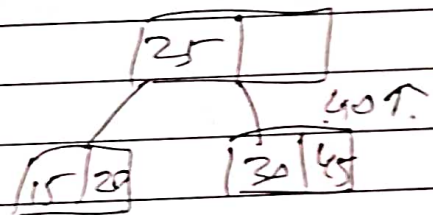
20, 30, 25, 15, 45,
 40, 10

max(mem) of new
 node created = level + 1
 after split

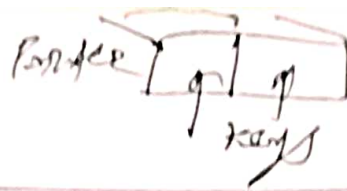
25 ↑
 20 | 30



↑ = split
 when
 overflow



Root at level = 0
By default



Ex

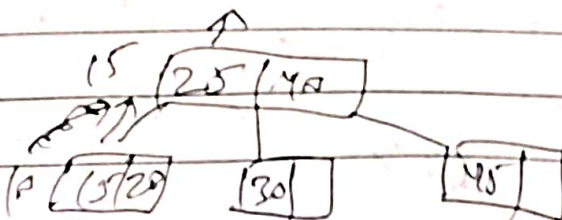
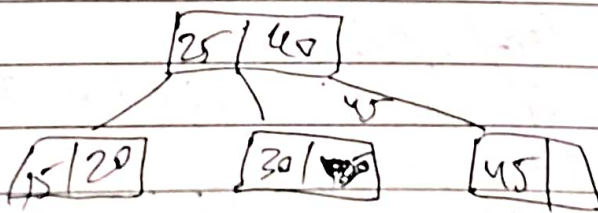
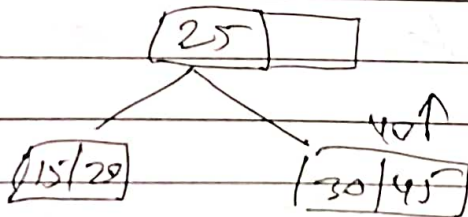
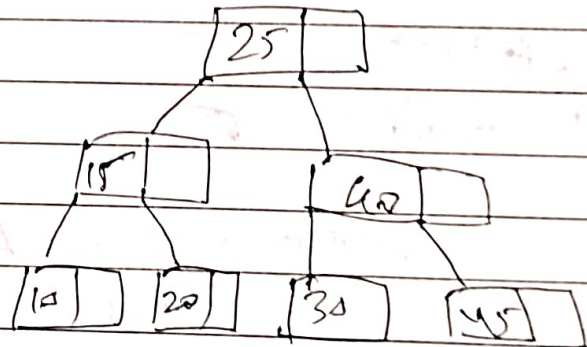
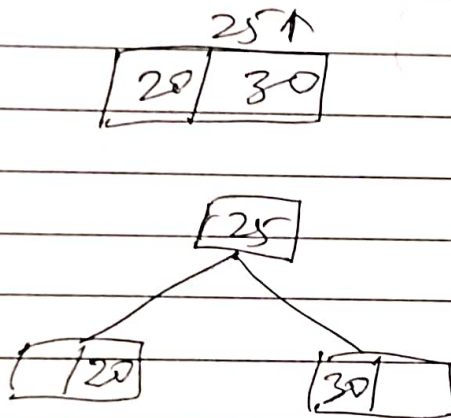
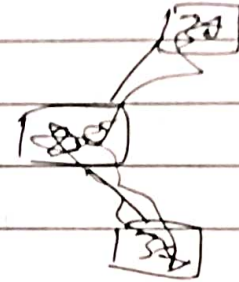
Insert the following keys into B-Tree of order 3.

Keys = 20, 30, 25, 15, 45, 40, 10

Soln $\Rightarrow$

max = 3 pointer,
(3-1) keys = 2

min = 2 pointer, 1 key



num of leaf splits = 3

— II — root splits = 2

— II — Internal splits = 1

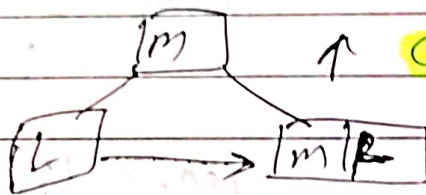
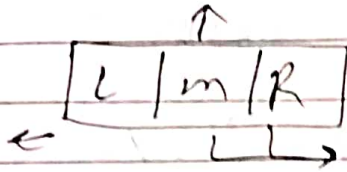
— II — total splits = 4

— II — levels = 2

— II — nodes = 7

— II — leaf node = 4

— II — Internal nodes = 3

B+ Tree Insertion

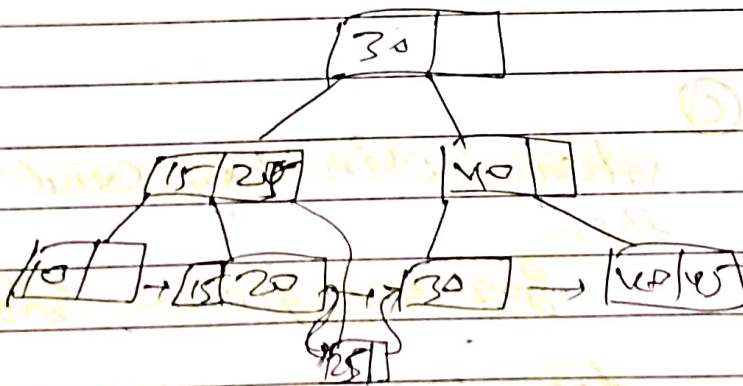
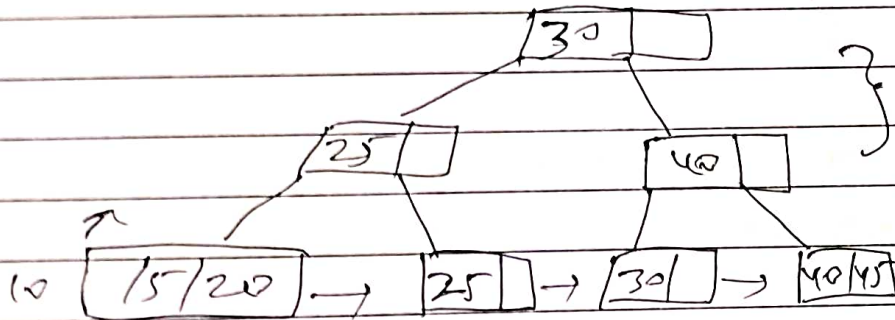
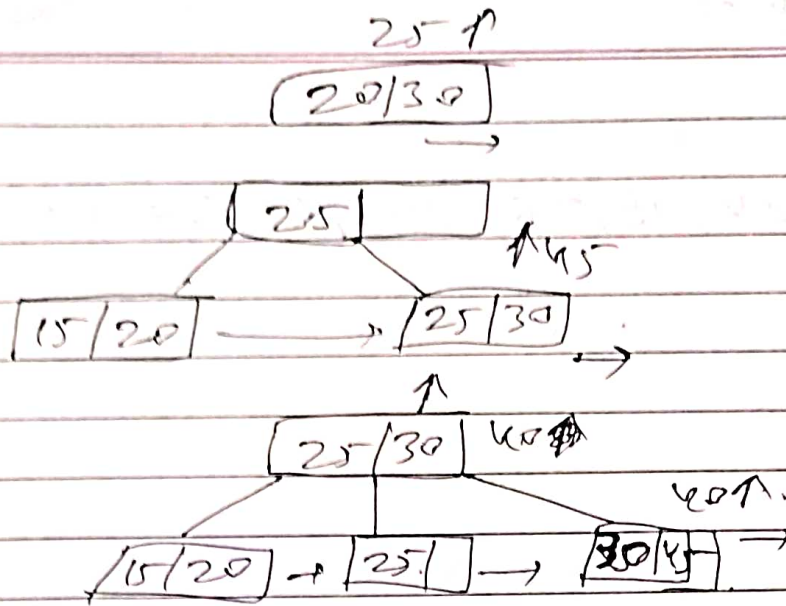
↑ copy of middle go to up

& copy of middle also go to Right side

→ The leaf node split ~~line~~ copy of middle element move to its higher level & all the keys less than the middle move to left subtree & all keys & greater than or equal to keys move to right side

Note → here Internal node split same as

B Tree insertion



num (root) split = 2

- 1 - leaf split = 48

- 1 - total split = 5

root at level 0
root = 1 ÷ level = 2

- 1 - full nodes = 3

- 1 - half nodes = 4

- 1 - Internal nodes = 3

Root nodes = 30



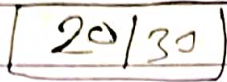
Ex → Insert following keys in B+ Tree of order 3.

20, 30, 25, 15, 45, 40, 10

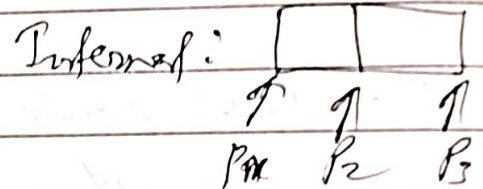
Soln →

↑ 25

order = 3



leaf: $[k_1/k_2]$

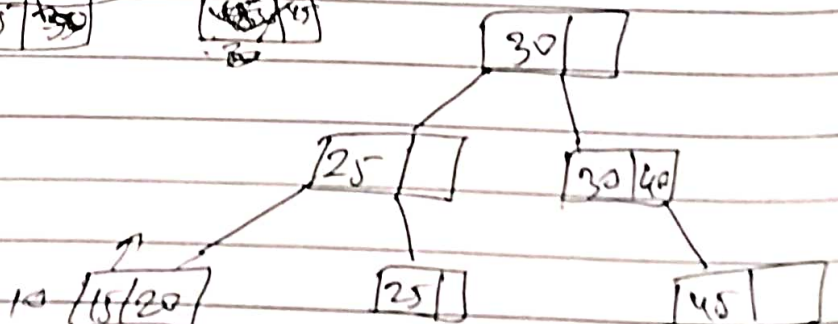
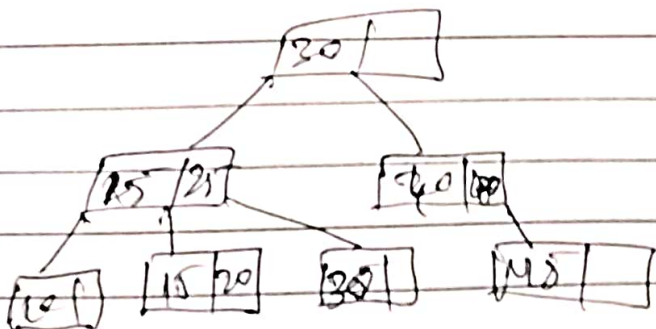
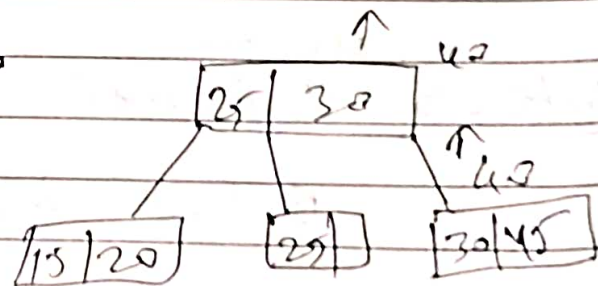
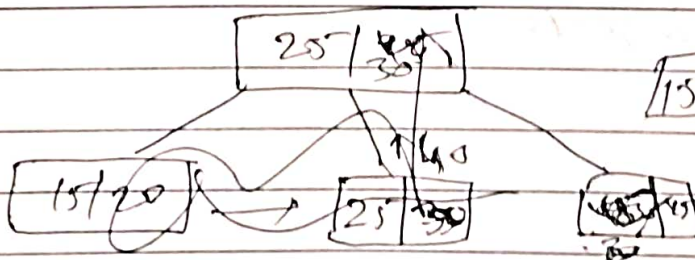
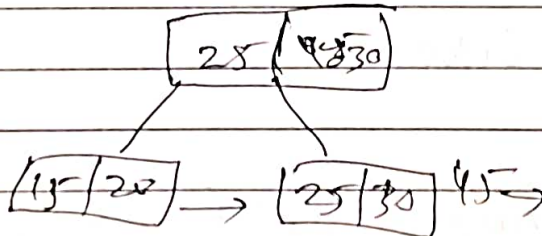
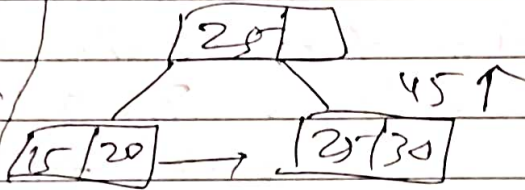
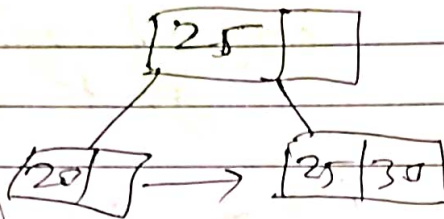


order = n

Internal node = n
order

leaf order = (n-1)

cost of node
200 only if
leaf level
not in internal



B Tree Deletion

Page No. _____

Date: ____/____/____

Pointers = Order = 3

max: 3 ptr, 2 key

min: 2 ptr, 1 key

Case 1 → delete from leaf

delete: 50

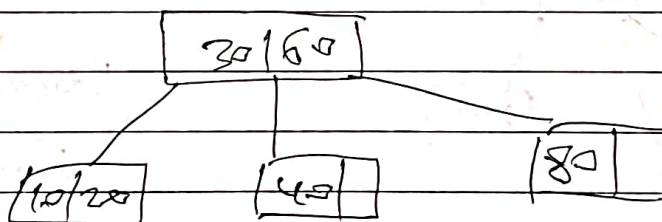
no Problem

as min key = 1

no adjustment req

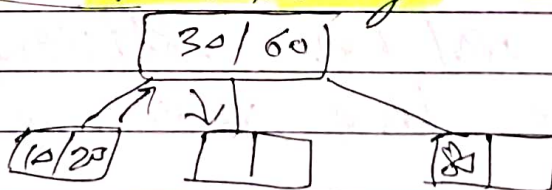
$$\left\{ \begin{array}{l} \text{ptr} = \left\lceil \frac{n+1}{2} \right\rceil \\ \text{key} = \left\lceil \frac{n}{2} \right\rceil + 1 \end{array} \right.$$

→ If node has more than min case then deletion of key not req modify



Case 2 delete from leaf violating min key condition

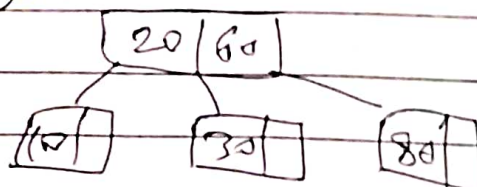
delete: 40



Ref A max key
Borrow from
left as it has
more keys.

key = no key = less than min

Ref B Borrow from right or
left sibling
through its
parent node



same way
B & B+ tree

Page No. _____

Date: ____/____/____

Ex

order = 3

keys = 24, 18, 36, 12, 45, 30, 9, 3

point (c)

when min key condn violates
then

Borrow from siblings

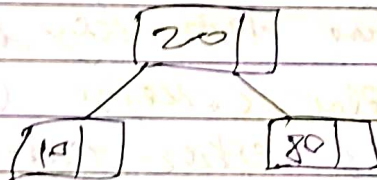
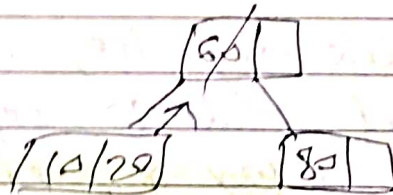
depending on which sibling
(either left or right)

has more than (keys).

(Internal)^{node}

Case 4 $\Rightarrow$ Root delete

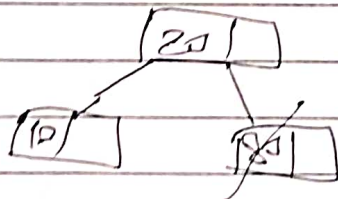
delete: 50



If root is to be deleted & it has min key then borrow

from child either left or right subtree & take largest key from left &

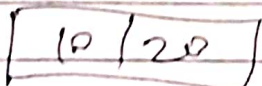
Case 5 $\rightarrow$ when each node has min key & there is possibility of violating min key condⁿ
delete: 80



Smallest key from right subtree

* collapse level = merge nodes

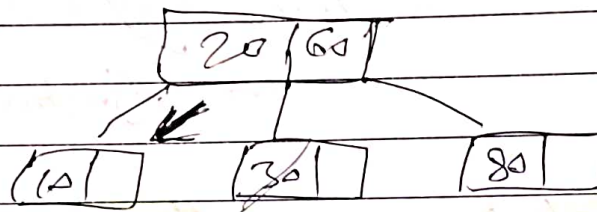
When parents & sibling wif have sufficient keys then merge remaining nodes (parent & siblings)



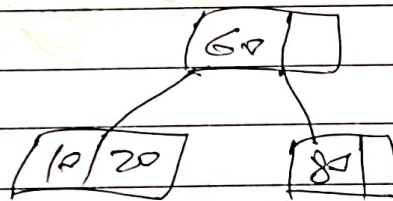
to make one node

→ deleteⁿ falls below min key than
Borrow key from either left or right
siblings.

Case 3 ⇒ delete from leaf when either siblings doesn't
have more than min
key
delete: 30



if left &
right sibling
has minimum
key than



Borrow from
Root (Parent)

when siblings not have sufficient keys &
unable to lend value key &

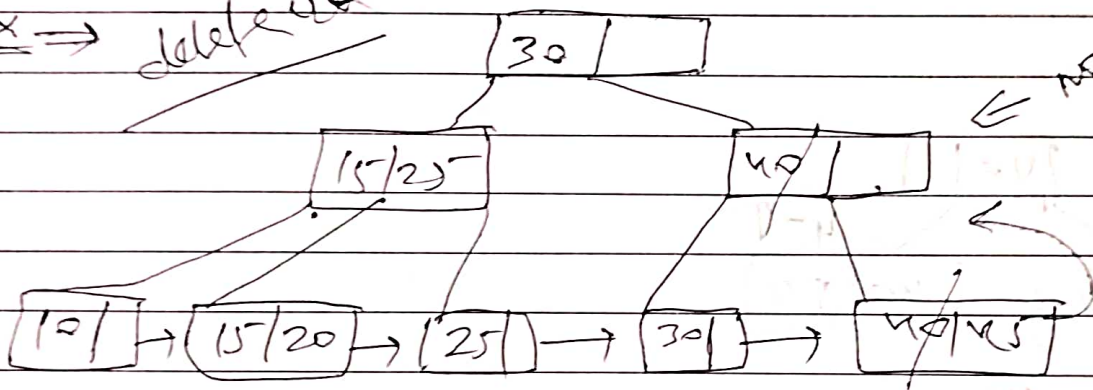
• Borrow from Parent & merge with
either left or right sibling

B+ tree deletion

↳ B+ deletion, not possible to have
Borrow keys from parents

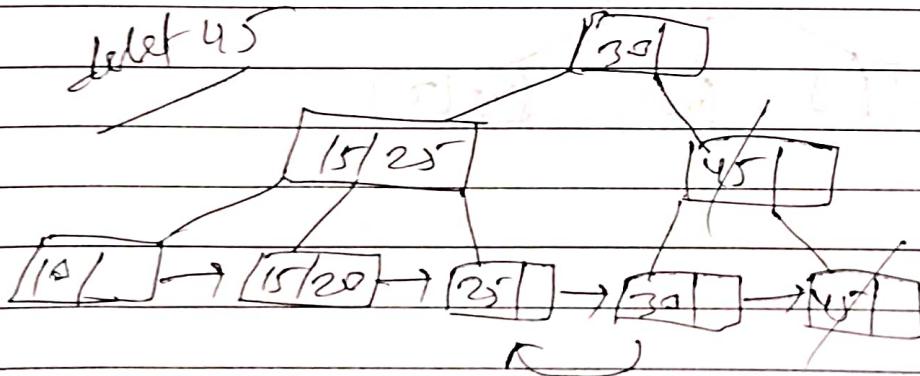
∴ directly merge nodes

Ex ⇒ delete 40



← no value has
take largest from
right subtree

delete 45

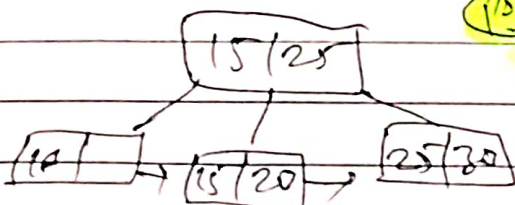


merge two
Adjacent
siblings

Point C root have only left subtree
which is not possible.

(Point D)

shift root
towards right
& keep its one copy only



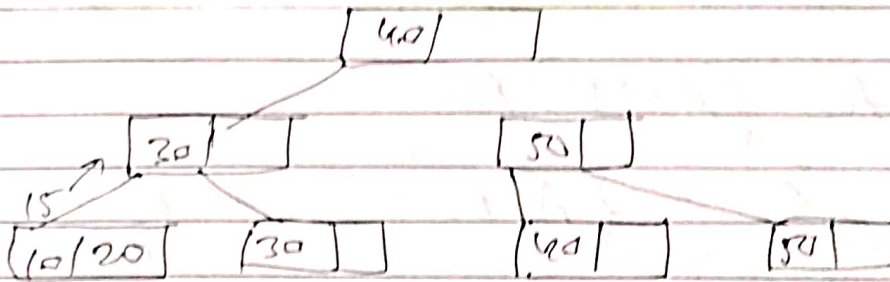
B+ Tree deletion

Points

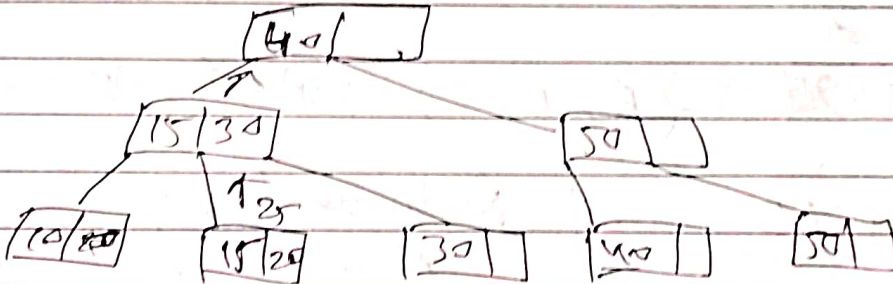
(A) when delete any key, if it's copy present delete copy as well
(key & its copy both deleted)

(B) If node has min key during deletion, then apply collapse level (merge nodes) & merge either toward left or right for siblings.

Q. 20

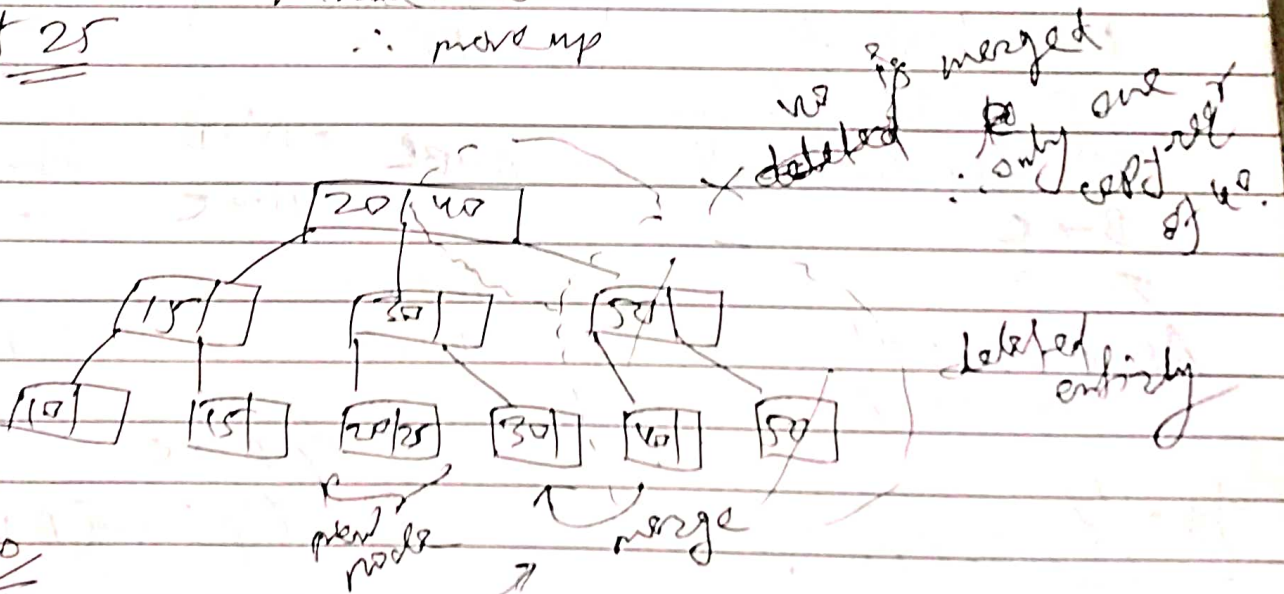


insert 15

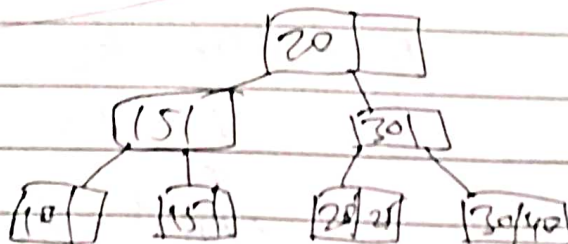


insert 25

middle = 20
∴ move up



delete = 50

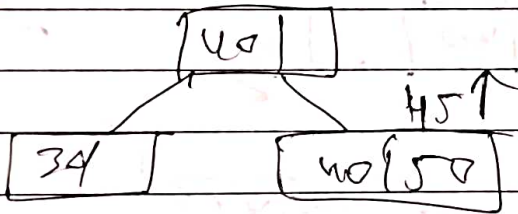


Q. 17max, key = 5

$$\text{pointer} = \text{key} + 1 = 5 + 1 = 6$$

$$\frac{\text{min} \text{ key}}{\text{pointer}} = \frac{6}{2} = 3$$

$$\rightarrow \text{key} = 3 - 1 = 2 \quad \{ (\text{pointer} - 1) \}$$

Q. 18

insert 45



Essential = Minimal = Canonical = Irreducible set of FD
FD's cover

Page No. _____

Date: ____/____/____

A finding Minimal or Canonical cover

Ex 1) $R = \{A, B, C\}$, $FD = \{A \rightarrow BC, B \rightarrow C, A \rightarrow B, AB \rightarrow C\}$
minimize the set of FD's.

Solⁿ $\Rightarrow$ ① Identify Redundant FD.

$F: (X \rightarrow Y)$

$\hookrightarrow$ Redundant then it must derive from other FD's.
 $\hookrightarrow$ whose removal not affect. ~~to~~^{it} derive from remaining FD's.

② Now Take 1st FD & check ^{whether} ~~whether~~ it is Redundant or not?

$A \rightarrow BC$

$\uparrow$
Take $A^+ = A$'s closure = $\{A, B, C\}$
without using this FD

③

Find A 's closure using other remaining FD's.

$\Rightarrow A^+ = ABC$

$\therefore A \rightarrow BC = \text{Redundant}$

now check for 2nd FD

$B \rightarrow C$

$\uparrow$

Take $B^+ = B$'s closure = $\{B, C\}$ = Essential

other FD doesn't derive $B \rightarrow C$

Every Attribute is only
closure of itself, if it doesn't
derive from other FD

Page No. _____

Date: ____/____/____

Now check for $(A \rightarrow B)$ FD

Take $A^+ = \{A, B, C\} \Rightarrow A^+ = ABC$
 $A \rightarrow B \Leftarrow$

we can derive it from other
FD. $\therefore A \rightarrow B$ is Redundant

Now check
FD $(AB \rightarrow C)$

Take $(AB)^+ = \{A, B, C\} \Rightarrow (AB)^+ = ABC = \text{Redundant}$

$\therefore$ Essential FD

$B \rightarrow C$

Now check for
FD which derive
All element with

$B \rightarrow C$

As it is not
derive A

Finally
Essential FD

$\{ B \rightarrow C, A \rightarrow B \}$

Redundant FD

$A \rightarrow BC$

$A \rightarrow B$

$AB \rightarrow C$

Now
check here
for Redundant
or closure

$A \rightarrow B \rightarrow A \rightarrow C$
 $A \rightarrow B$

Find A^+
 $= A, B, C$

$A \rightarrow B$

$AB \rightarrow C$

$\therefore A \rightarrow B = \text{Redun}$
Element it

Now, $A \rightarrow C$

$A^+ = \{A, B, C\}$

$A \rightarrow C$ Redundant
eliminate it

Now check for

$AB \rightarrow C$

$(AB)^+ = ABC \therefore \text{Redundant}$
eliminate it

Ex
 $R = \{A, B, C, D, E\}$ The corresponding

FD = $\{A \rightarrow BC, CD \rightarrow E, B \rightarrow D, E \rightarrow A\}$

Soln

~~use left reduced~~

~~(make LHS = 1 attribute)~~

~~Right Reduced~~

~~(make RHS = 1 attribute)~~

~~$CD \rightarrow E$
 $C \rightarrow E$
 $D \rightarrow E$~~

~~$A \rightarrow BC$~~

~~$A \rightarrow B$~~

~~$A \rightarrow C$~~

$\therefore A \rightarrow BE$

$A \rightarrow C$

$C \rightarrow E$

$D \rightarrow E$

$B \rightarrow D$

$E \rightarrow A$

$E \rightarrow A$

$A \rightarrow B$

$B \rightarrow D$

$\rightarrow$ By transitivity

$A \rightarrow D$

$\rightarrow A \rightarrow C$

$\Rightarrow A \rightarrow CD$

$\therefore A \rightarrow CD \rightarrow E$

$\therefore A \rightarrow E$

$\therefore$ All FD's are essential

$\therefore$ minimal set = irreducible = minimal cover

$\therefore \{A \rightarrow B, A \rightarrow C, A \rightarrow E, B \rightarrow D, E \rightarrow A\}$

Wrong

Essential

$$\begin{aligned} A \rightarrow B &= A^+ \{A, C\} \\ A \rightarrow C &= A^+ \{A, B, D\} \\ CD \rightarrow E &= CD^+ \{C, D\} \\ B \rightarrow D &= B^+ \{B\} \\ E \rightarrow A &= E^+ \{E\} \end{aligned}$$

All FD's are essential

"E-R diagram Components"

Page No. _____

Date: ____/____/____

[E] → Entity set

[E] → Weak Entity set

◇ → Relationship set

◇ → Identifying Relationship set
for weak entity set

A → Attributes

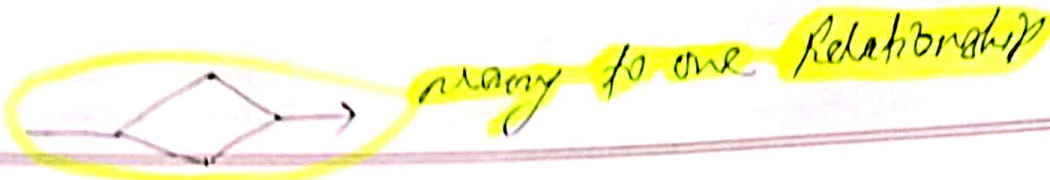
A → Multivalued Attributes

Derived Attribute →

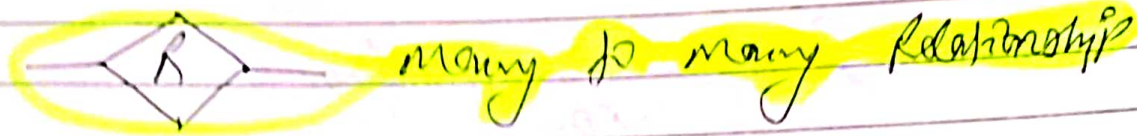
Determining
Attribute
of weak
entity set

◇ → [E] → Total Participation of entity set
in Relationship

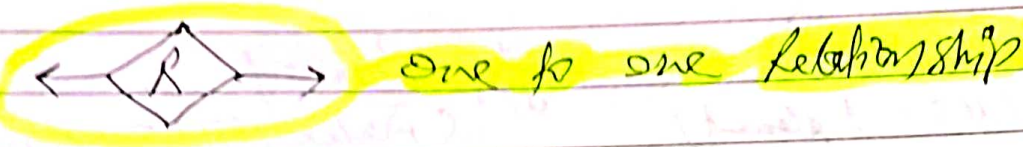
◇ → [E] → Partial participation



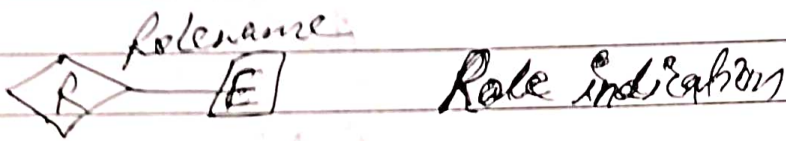
many to one Relationship



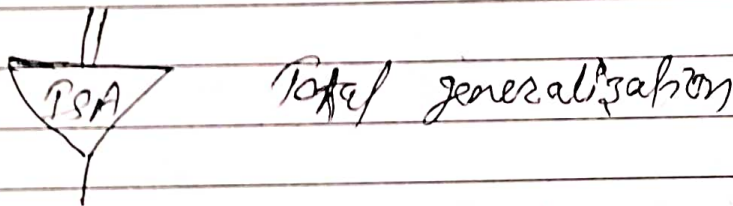
many to many Relationship



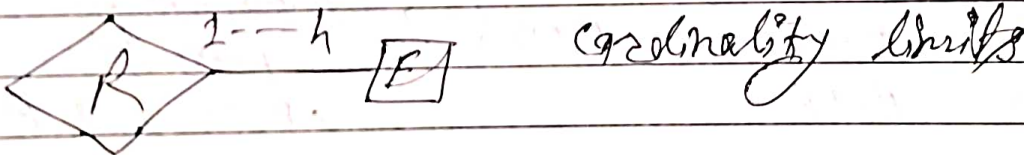
one to one Relationship



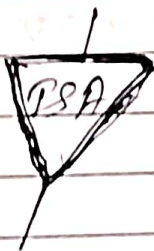
Role indication



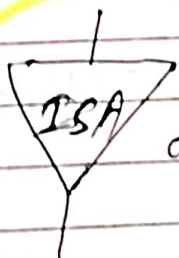
Total generalization



cardinality limits



ISA Specialization or Generalization

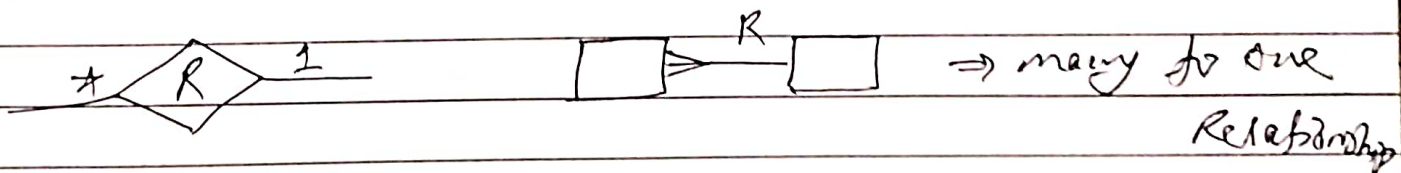
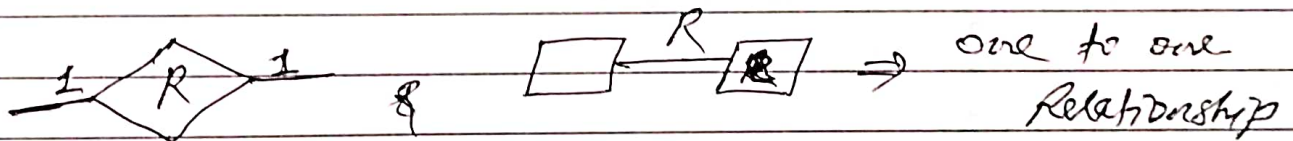
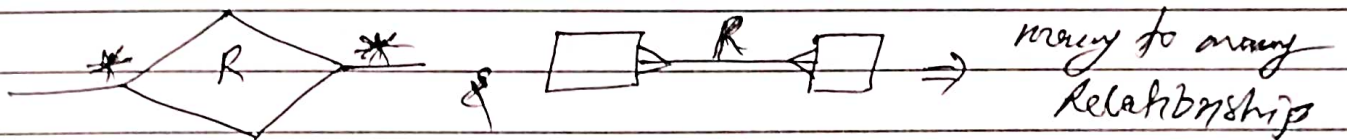
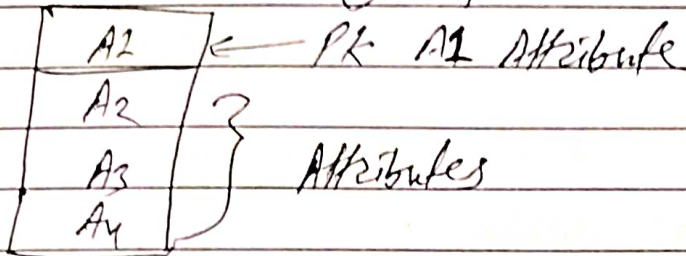


disjoint

Disjoint Generalization

"Alternative E-R Notations"

$E \rightarrow$ Entity set



"Types of Attributes" (Attr)

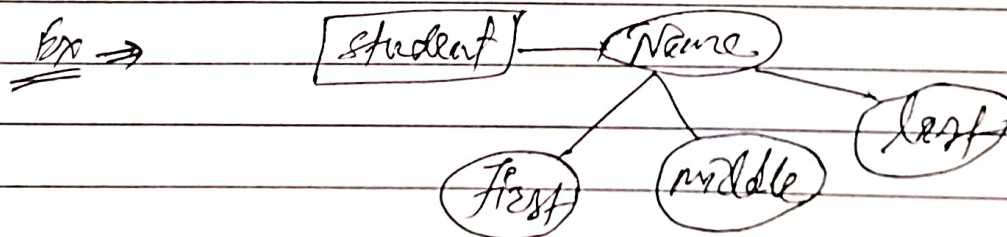
① Simple Attribute

↳ only Atomic values stored
↳ which can't divide these value further

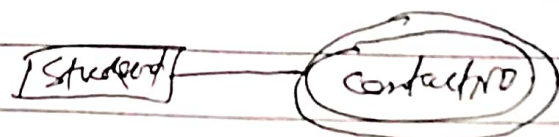
Ex ⇒ student, (phone no.) → Atomic value 7588017470
Not 72558...
(Age) → Atomic value
(can't divide (25 yr))
2 & 5 divide
× possible

② Composite Attribute

↳ which can be divided into smaller sub parts



③ Multi valued Attribute



ContactNo → 758801747
 → 9822108015

"Introduction of DBMS"

Page No. _____
Date: 22/05/21

Data $\Rightarrow$ worth of millions or billions

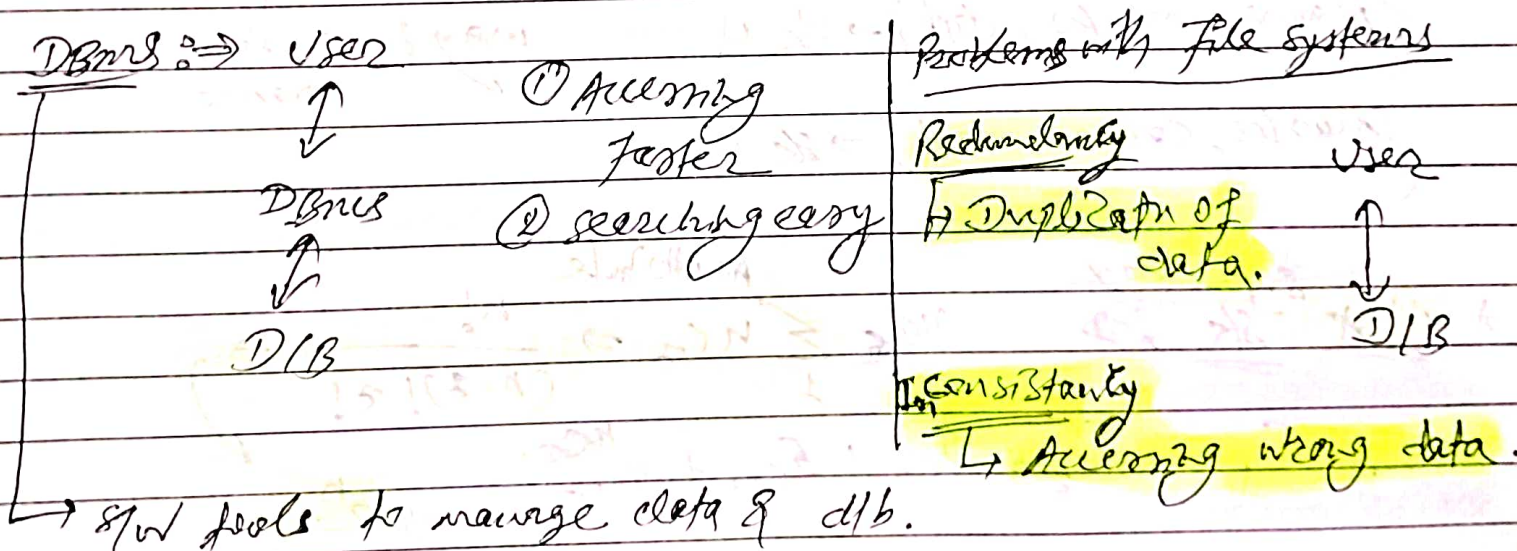
$\hookrightarrow$ facts which stored

text, num, videos, speech, audio, images.

DB $\Rightarrow$ collⁿ of related data that represent some real world entities.

$\rightarrow$ change in real world entity, changes in db.

Info $\Rightarrow$ meaningful / processed data / useful infⁿ in specific seg^t



$\rightarrow$ For which type of data & that type of dbms seg^t

diff. data $\Rightarrow$ diff dbms seg^t.

App^n of DBMS

→ Hospital
→ Railways

DBMS → data orgn as hierarchical form

* RDBMS

→ data stored in form of tables/relations

DBMS RDBMS

Key = Super key } Both are one meaning

Page No. _____

Date: ____/____/____

Keys in DBMS

fields = columns = Attributes

Student Relation

Sid	Name	Marks	Dept	Course
1	a	78	CS	C1
2	b	60	EE	C2
3	a	78	CS	C3
4	d	60	EE	C4
5	e	80	IT	C5

Need of key

Identify each records uniquely

To access data effectively &

Avoid ambiguity or confusion of accessing duplicate tuples

→ {dept + course} unique key
 $\left. \begin{matrix} CS + C1 \\ CS + C3 \end{matrix} \right\}$ ← unique value

⇒ we can combine keys to uniquely identify records.

→ {marks + dept} × unique
 $\left. \begin{matrix} 78 + CS \\ 78 + C3 \end{matrix} \right\}$ ← duplicate value

∴ It is Not key

Comb of Attributes are
 ⇒ {Name + marks + Dept + Course} unique
 ∴ Key

Domain = type of value that an Attribute must have

Constraint = Rules that every Attribute must follow

tuple = Row = Record

Page No. _____

Date: ____/____/____

keys $\Rightarrow$ It is an attributes or set of attributes that uniquely identify each records of relations.

Super key $\Rightarrow$ It is the key

$\{sid\} \rightarrow sk \checkmark \rightarrow$ unique

$\{sid\} + \{sname\} \rightarrow sk \checkmark$

1 + a $\rightarrow sid=1, sname=a$

3 + a $\rightarrow sid=3$

student name = a

$\{Name, marks, dept\} \rightarrow sk \checkmark$

multiple students may have same name

$\{marks, course, dept\} \rightarrow sk \checkmark$

super key

All Attribute

$$\sum_{i=1}^n nC_i = \frac{n!}{(n-i)! i!} \Rightarrow$$

$$= 5C_1 + 5C_2 + 5C_3 + 5C_4 + 5C_5 \Rightarrow 2^n - 1$$

$$= 5 + 10 + 10 + 5 + 1 = 31$$

5 = Total Attributes

1 $\Rightarrow sid$

2 $\Rightarrow sid + sname$

3 $\Rightarrow name + mark + dept$

4 $\Rightarrow sid + name + mark + dept$, 5 $\Rightarrow sid + name + marks + dept$

We have to check

not in All cases

gives correct num of sk.

the
Primary $\rightarrow$ order doesn't matter

Page No. _____

Date: / /

Ex $\Rightarrow R(A, B, C, D)$

1	1	5	1
2	1	7	1
3	1	7	1
4	2	7	1
5	2	5	1
6	2	5	2

$\Rightarrow$ max

$$\text{Total num (SK)} = 2^4 - 1$$

$$= 2^4 - 1 = 15$$

$B \neq SK$
 $C \neq SK$
 $D \neq SK$

} Not unique
identity each
records

$$SK_1 = A$$

$$SK_2 = A + B$$

$$SK_3 = A + B + C$$

$\{A, AB, AC, AD\}$
 ~~$\{A, B, C, D\}$~~ , ABD ,
 $ABC, ACD, ABCD$

(Remaining attributes)

$$\text{num (SK)} = 8$$

with 'A'

$$\text{num (SK)} = 2$$

with 'A'

$\rightarrow CD \neq SK$

$\rightarrow BC \neq SK$ not unique
has duplicates
(1, 7)

$\rightarrow BD \neq SK$ not unique
has duplicate
(1, 1)

$\rightarrow BCD \neq SK$ not unique
has duplicates
(1, 7, 1)

if larger set of Attributes
is not ^{super} key, then its
subset never be SK

$BCD \neq SK$

$\therefore BC$ & $CD \neq SK$
BD

$BC \subseteq$
 $CC \subseteq$
 $DC \subseteq$

} BCD

$\therefore$ Not SK

$BC \subseteq$
 $CD \subseteq$
 $BDC \subseteq$

} BCD

If Any Attribute is ^{super} key then all
its ^{super} comb are keys

ck is form only from sk

Not sk = Not ck

Page No. 4 for All
Date: / / Attendance

Candidate key (ck)

→ sk whose proper subset is not sk.
→ minimal sk

R(A, B, C)

1 1 1

2 1 2

3 2 1

4 2 2

Here, R(A, B, C)

sk $\Rightarrow$ A, AB,

AC, BC

ABC

Not sk

Bx

Cx

for single attribute, it is not proper subset of itself
- no violation

→ Now, check "ABC" is candidate keys?

"ABC" its proper subset is sk $\Rightarrow$ $\therefore$ violate the condn for

A, B, C, AB, AC, BC

$\therefore$ ABC $\neq$ ck

But $\nearrow$ sk

Candidate key

$\Rightarrow$ Not a single subset will be a proper subset for candidate key

→ Now, check "AC" is ck?

A C

A is sk $\therefore$ AC $\neq$ ck

→ Now check "A" is ck?

Yes not has any subset to violate condn

$\therefore$ A = ck

Proper Subset = Both the sets not have equal num of elements and same elements

Page No. _____

Date: ____/____/____

Symbol
Subset ($\subseteq$)

$$S_1 = \{1, 2, 3\}$$

$$S_2 = \{1, 2\}$$

$$S_2 \subseteq S_1$$

All elements of S_2 , must be in S_1

$$\therefore S_2 \subseteq S_1$$

$$\{1, 2\} \subseteq \{1, 2, 3\}$$

Symbol
Proper subset ($\subset$)

Case - I

S_1

$$S_1 = \{1, 2, 3\}$$

$$S_2 = \{1, 2\}$$

must satisfy 2 condⁿ

$$\text{① } S_2 \subseteq S_1 \checkmark$$

$$\text{② } S_1 \not\subseteq S_2 \checkmark$$

$\therefore$ Here S_2 is proper subset of S_1 .

$$S_2 \subset S_1 \checkmark$$

Case - II

$$S_1 = \{1, 2, 3\}$$

$$S_2 = \{1, 3, 2\}$$

Check 2 condⁿ

$$\text{① } S_2 \subseteq S_1 \checkmark$$

$$\text{② } S_1 \subseteq S_2 \checkmark$$

$\therefore$ It is not proper subset of $S_2 \subset S_1$.

Minimal = As min as possible

Page No. _____

Date: ____/____/____

Rule $\Rightarrow$ Empty set of Attribute can't be a sk

Condition of Candidate key

$\hookrightarrow$ minimal sk

Proof $\Rightarrow ABC = sk$,

$R(A, B, C)$		
1	1	1
2	1	2
3	2	1
4	2	2

Remove any Attribute,

$AB \neq$

$AB \rightarrow B$ is sk, yes

Remove Attribute

$AC \neq$

$A \rightarrow B$ is sk, yes

Remove A not possible

$BC = sk$

Remove B

$C \neq sk$

Either Remove

C

$B \neq sk$

$\therefore BC = \text{minimal sk}$

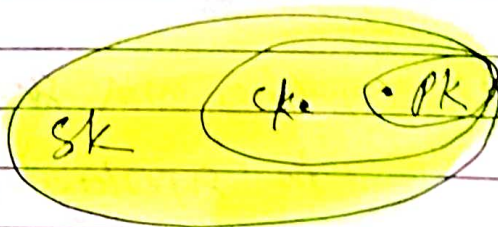
$\therefore A = \text{minimal sk}$

$\therefore A = ck$

As Empty set $\neq sk$

$\therefore BC = sk$

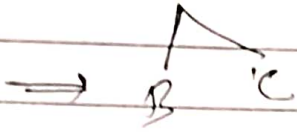
for single Attribute, not proper subset
 $\therefore$ no violation



$\nRightarrow$ Every ck is sk but vice versa is not possible

check

"BC" is ck?

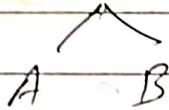


B is not sk
C is not sk

$\therefore BC \neq ck$

R	ABC
1	111
2	12
3	21
4	22

→ now check "AB" is ck?



A = sk

$\therefore AB \neq ck$

note: A, B & C

individually not a key
as they have duplicate
records,

→ check B is ^{not} ck = ?

↓

yes, B is ck, as it not has any subset.

→ check C is ^{not} ck = ?

↓

yes, C is ck as it not has any subset.

Candidate

key $\Rightarrow ck(R(A, B, C))$

A, BC = (2) $\leftarrow \text{max}(ck)$

Alternate keys = keys other than Primary key

Page No. _____

Date: ____/____/____

→ If Any one pair of SK is proper subset, then
(or Attribute)
also that Attribute / SK
is not a CK.

Ex → ABD ⇒ SK

A ⇒ is SK

∴ ABD ≠ CK

Refer ~~the~~
R(A, B, C)

Example of
Previous page

A Primary key ⇒ Any one CK is selected as PK,
But it should not be NULL.

→ PK has multiple Attributes

Ex → We can pick any ^{8CK} keys A, BC
as PK.

If A = PK, Remaining BC = Alternate key

If BC = PK, Remaining A = Alternate key

Refer R(A, B, C)

Example of Previous
page

→ Each Relation has

more than 1 $\swarrow$ SK
 $\searrow$ CK

⇒ multiple Attribute Allow
 for creating $\swarrow$ SK
 $\searrow$ CK
 $\searrow$ PK

Ex ⇒ $R(A, B, C)$

if A is CK then find min(SK)?

Solⁿ
 ⇒

A is CK $\therefore A = SK \checkmark$

$AB \checkmark = SK \quad | \quad AC \checkmark = SK \quad | \quad ABC \checkmark = SK$

$\therefore \min(SK) = \underline{\underline{4}}$

Note ⇒ Any Attributes combine with SK will result
 in superkey (SK)

Note ⇒ if a Attribute = CK, it must be a SK.

Ex $\Rightarrow R(A, B, C)$

$AC = CK$ then $\text{num}(SK) = ?$

Soln $\Rightarrow AC = CK$

$\therefore AC = SK$ $\left\{ \begin{array}{l} \text{As each } CK = SK \text{ By} \\ \text{Rule} \end{array} \right.$

$\therefore ACB = SK$

$\therefore \text{num}(SK) = \underline{\underline{2}}$

Both $\{ABC$

are

same, As ordering Not matters.

$SK = \{AC, ACB\}$

Check A is SK, Ans = NO, or AC is CK & if is minimal

$\therefore$ A alone or B alone is not a SK.

Every Candidate ~~key~~
key

$CK = \text{minimal} = \text{Irreducible}$

"Functional dependency"

determinant

(→)

some depends on something

dependent

FD: $x \rightarrow y$

{ x determines y
read as

→ from value of
x find
value of y.

x	y
1	1
2	1
3	2
4	3

Ex → FD: $x \rightarrow y$ Case = I → $x = 2$ then $y = 1$ ✓ FD

x	y
1	1
2	1
3	2
4	3
2	5

Case = II

if $x = 2$

then $y = \begin{matrix} 1 \\ 5 \end{matrix}$ } Confusion/Ambiguity
 x not possible ∴ not FD

Combⁿ of $x \rightarrow y$

must be unique

to be FD

FDdefn ⇒ if in 2 tuple, $(t_1.x = t_2.x)$

then y value must be same.
 $(t_1.y = t_2.y)$

∴ FD exists ✓

checking for FDFD: $x \rightarrow y$ Exist?

	x	y
$t_1 \rightarrow$	1	1
$t_2 \rightarrow$	2	1
	3	2
	4	3
	2	5

check $t_1.x = t_2.x$

$1 \neq 2$

no need to check y value

check $t_1.x = t_2.x$
 $2 \neq 3$ no need to check
y value

	x	y
$t_1 \rightarrow$	1	1
$t_2 \rightarrow$	2	1
	3	2
	4	3
	2	5

similarly do it for all,
if $t_1.x = t_2.x$ then check $t_1.y = t_2.y$ check $t_1.x = t_2.x$

$2 = 2$

then check y value

$t_1.y = t_2.y$

$1 \neq 5$

Not Equal means FD Doesn't Exist

* NO FD: $x \rightarrow y$

for given Relation

may be one or
set of attributes

Unique = distinct

Page No. _____

Date: ____/____/____

FD: $X \rightarrow Y$

check

$$f^D.X = f^D.X$$

unique
or distinct

R	
X	Y
1	1
2	1
3	2
4	3

if X value are unique, then no need to check
Y value

And hence FD condn not violate

$\{ f^D.Y = f^D.Y \}$ not even
checked

In
such case

FD Always exists

Ex: $\Rightarrow$ check FD ~~Exists~~ or Not?

$A \rightarrow B$ $\frac{?}{?}$ Exists

or Not

t_1
 t_2

A	B
1	2
2	3
3	4
4	2
1	2

t_1

t_2

Soln

~~Ans~~

$\rightarrow$ if $A \rightarrow t_1.x \neq t_2.x = \text{Unique}$, No. Need to check B's value

Summary check for all $\rightarrow$

	A	B
$\rightarrow t_1$	1	2
$\rightarrow t_2$	1	2

for $A \rightarrow t_1.x = t_2.x = \text{duplicate value}$

$\therefore$ check for B's value

for $B \rightarrow t_1.y = t_2.y = \text{same values}$

$\therefore$ Even though A has duplicate tuple '1' but in B it has same value

'FD Exists'

Types of FD

- Trivial
- Non-Trivial
- Semi-Trivial FD
- multivalued
- Transitive

① Trivial FD (TFD)

- ① FD: $X \rightarrow Y$ = Trivial FD
 $\left\{ \begin{array}{l} \text{if } Y \subseteq X \end{array} \right.$
- ② All $X \rightarrow X$ = Trivial FD

Trivial FDs
are Always
Valid.
No need to
check in
Relation.

Ex: $RNO \rightarrow RNO \checkmark$ TFD

$\{RNO, Name\} \rightarrow Name \checkmark$ TFD

$\{RNO, Name\} \rightarrow RNO \checkmark$ TFD

$\{Y \subseteq X\}$

RNO	Name	marks	Dept

③ Semi Functional FD (SFD)FD: $X \rightarrow Y$ iff $Y \not\subseteq X$ Ex $\Rightarrow \{Rno, Name\} \rightarrow \{Name, marks\}$ $\therefore$ SFD ✓① $\{Rno, Name\} \cap \{marks\} = \emptyset$
= NTFD $marks \subseteq \{Rno, Name\}$ $\therefore$ SFD② $\{Rno, Name\} \cap \{Name, marks\} \neq \emptyset$ $\therefore$ Not NTFD② Non-Functional FD (NTFD)FD: $X \rightarrow Y$ iff $X \cap Y = \emptyset$ $\Rightarrow$ we have to checkfor ^{NT} FDs valid orInvalid in
RelationEx $\Rightarrow Rno \rightarrow marks = \text{NTFD}$ As $Rno \cap marks = \emptyset$

Nothing common betn them

But we have to check it again at Relation

Ex $\Rightarrow \{Rno, marks\} \rightarrow \{Name\}$ $\rightarrow$ SFD ✓ $\{as\} \{Name\} \not\subseteq \{Rno, marks\}$ $\rightarrow$ Non TFD ✓ $\{as\} \{Name\} \cap \{Rno, marks\} = \emptyset$ $\rightarrow$ TFD $\times$ As nothing common & not subset

→ To check FD Exists or Not By using Properties
 "Armstrong's Axioms / Inference Rules"

Page No. _____

Date: ____/____/____

Properties

→ To compute something

① Reflexivity:

$$X \rightarrow X$$

$$X \rightarrow Y, Y \subseteq X$$

Ex $\Rightarrow$ Rno $\rightarrow$ Rno ✓
 Name $\rightarrow$ Name ✓

② Transitivity:

if $(X \rightarrow Y \ \& \ Y \rightarrow Z)$
 then only $X \rightarrow Z$

Both condⁿ must follow

Ex $\Rightarrow$ Name $\rightarrow$ marks ✓ FD
 marks $\rightarrow$ Dept ✓ FD

$\therefore$ Name $\rightarrow$ Dept

By X rule

Rno	Name	Marks	Dept	Course
1	a	78	CS	C1
2	b	60	EE	C1
3	a	78	CS	C2
4	b	60	EE	C3
5	c	80	IT	C2

③ Augmentation: if $X \rightarrow Y$ then $XA \rightarrow YA = \text{Exists}$
 ↳ To Add something

Ex $\Rightarrow$ Rno $\rightarrow$ Name ✓ FD
 Add 'marks'

$\therefore$ Rno, marks $\rightarrow$ Rno, Name, marks ✓

Rno = Unique
 $\therefore$ FD
 Rno, Name
 is valid

$\{Name, marks\} \rightarrow \{Dept, Course\}$

$\therefore \{Name, marks\} \rightarrow Dept \ \& \ \{Name, marks\} \rightarrow Course$

Page No. _____

④ Union : if $(x \rightarrow y \ \& \ x \rightarrow z)$ then $x \rightarrow yz$

Ex $\rightarrow RNO \rightarrow Name \ \& \ RNO \rightarrow marks$

$\therefore RNO \rightarrow \{Name, marks\}$

⑤ Decomposition / splitting

if $(x \rightarrow yz)$ then $x \rightarrow y$
 $x \rightarrow z$

Ex $\rightarrow RNO \rightarrow \{Name, marks\}$ ✓

~~to~~ $\therefore RNO \rightarrow Name \ \& \ RNO \rightarrow marks$

Note $\rightarrow$ if $x \rightarrow y \rightarrow z$ then $x \rightarrow z$
 $y \rightarrow z$ } $\times$ Not possible
decomposable
Never be split/decomposed

$\Rightarrow$ split / decomposition always done at Righthand side
or with dependent